Problem C. Game

Input file: `standard input`
Output file: `standard output`

Alice and Bob play a game with a set initially containing all integers $1, 2, \ldots, 2n$.

The game proceeds in n rounds. In each round:

Alice's move: Alice selects two distinct numbers x and y from the current set, gives them to Bob and removes them from the set.

Bob's move: Bob chooses one of the two numbers x or y and adds it to his score. The remaining number is added to Alice's score.

Bob can not choose the larger of the two numbers in two consecutive rounds.

You are assigned the role of either Alice or Bob.

Your goal is to maximize your own score according to your role and the game rules.

Input

Your submission must not read from the standard input, print to the standard output, or interact with any other file.

You must implement **one** of the following functions, depending on the role you are assigned.

Please note that you should include **both** `void Alice(int n)` and `void Bob(int n)` in your submission or your solution will fail to compile. If you want to implement only one function, you may leave the body of the other function empty.

Function: `void Alice(int n)`

- n is half the size of the initial set.
- The initial set contains all integers from 1 to $2n$.
- Your function plays as **Alice**.
- The judge plays as **Bob**.

From this function, you must call the following function: `int AskBob(int x, int y)`

- This function gives Bob two distinct numbers x and y.
- Both x and y must not have been used in any previous call.
- The function returns the number chosen by Bob.
- You must call `AskBob` **exactly** n **times**.

Function: `void Bob(int n)`

- n is half the size of the initial set.
- The initial set contains all integers from 1 to $2n$.
- Your function plays as **Bob**.
- The judge plays as **Alice**.

From this function, you must call the following functions: `pair<int,int> AskAlice()` and `void AnswerToAlice(int c)`

- Function `AskAlice()` returns a pair of numbers (x, y) chosen by Alice.
- You must call the function `AskAlice()` first.
- After calling `AskAlice()`, you must call `AnswerToAlice(c)`, where:
  - $c \in x, y$
  - c is the number chosen by you
- You are **not allowed to choose the larger of the two numbers in two consecutive rounds**.
- After calling `AnswerToAlice`, you must call `AskAlice` again.
- You must call both `AskAlice` and `AnswerToAlice` **exactly n times**.

Scoring

- $2 \leq n \leq 1000$.

In this task, the grader is **adaptive**. It means that graders decisions are affected by your function calls.

This problem contains 7 subtasks.

Subtask	Additional Constraints	Points
0	Examples	0
1	$n \leq 3$, $k = 1$	4
2	$n \leq 3$, $k = 2$	4
3	$n \leq 8$, $k = 1$	10
4	$n \leq 8$, $k = 2$	10
5	$k = 1$, Bob always tries to choose the biggest number	8
6	$k = 2$	24
7	$k = 1$	40

If $k = 1$ you are asked to play as Alice. If $k = 2$ you are asked to play as Bob.

Your solution will receive the points for subtask if your score is at least S, where S — number of points you can get with the optimal solution, given that your opponent plays optimally too.

Note

Suppose that $n = 2$ and $k = 1$, i.e. you are asked to play as Alice. The grader starts your program by calling `Alice(2)`. Example of further interaction is below.

Call	Result	Comments
`AskBob(1, 2)`	2	Bob (judge interactor) chose 2. Now Bob's score is equal to 2, and Alice's score is equal to 1.
`AskBob(3, 4)`	3	Bob (judge interactor) chose 3. Now Bob's score is equal to 5, and Alice's score is equal to 5.

After the interaction, Bob's score is equal to 5, and Alice's score is equal to 5. Thus, your final score is equal to 5.

Suppose that $n = 2$ and $k = 2$, i.e. you are asked to play as Bob. The grader starts your program by calling `Bob(2)`. Example of further interaction is below.

Call	Result	Comments
`AskAlice()`	`(1, 2)`	Alice (judge interactor) chose 1 and 2. Now Bob has to choose one of those numbers.
`AnswerToAlice(2)`	`--`	Bob chose 2. Now Bob's score equals to 2, and Alice's score is equal to 1.
`AskAlice()`	`(4, 3)`	Alice (judge interactor) chose 4 and 3. Now Bob has to choose one of those numbers.
`AnswerToAlice(3)`	`--`	Bob chose 3. Now Bob's score equals to 5, and Alice's score is equal to 5.

Please note that your program cannot call `AnswerToAlice(4)` after second `AskAlice()` call, since Bob cannot choose biggest number in two consecutive rounds.

The sample grader reads the input in the following format:

- Line 1: n, k and T: n — Half the size of the initial array, k — either 1 (Alice) or 2 (Bob), and T — judge interactor type.

You can download an archive `game.zip` in the system that contains sample test cases, solutions and graders for all supported languages (C++17, Java 8 and Python 3).

For solutions in Java, file and class name have to be named `game.java` and `game` respectively. Please note that both have to be lowercase. You can refer to the sample Java code `game.java` from the archive.

You can refer to the sample Python code `game.py` from the archive.